Verifier-Guided Evaluation of LLM-Generated Network Configurations: a Paired Study with Shared Initial Candidates

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

***Abstract*—We preregistered and executed a paired confirmatory study (CAND-2026-001) that measured whether a verifier-triggered guarded pipeline (generate → validate → repair, <=1 repair) reduces deterministic-invariant violations relative to single-shot initial generation for intent→configuration NetOps tasks. Using a deterministic synthetic task generator and a deterministic network verifier, we ran 200 preregistered tasks under shared_initial_candidate semantics with the hosted model gpt-5-mini (execution/model_configuration.json records temperature = null/unset). Baseline configurations passed the verifier in 199/200 tasks (99.5%); the guarded pipeline passed 200/200 (100%). Paired absolute risk difference (guarded - baseline) = 0.005; contingency counts n11=199, n10=1, n01=0, n00=0; exact two-sided McNemar p = 1.0; paired-bootstrap 95% percentile CI = [0.00, 0.015] (10,000 resamples, seed 1729). The single guarded improvement arose from one verifier-triggered repair that corrected a multi-step ordering violation. We report preregistration, full execution accounting (201 model calls: 200 shared initial + 1 repair), paired analysis, representative diagnostic artifacts, and bounded limitations grounded in the archived execution bundle.**

I. Introduction

Generative large language models (LLMs) are increasingly proposed to translate high-level operator intent into device configurations for network operations (NetOps). Operational correctness for such workflows requires generated configurations to satisfy deterministic invariants (reachability, policy compliance, workflow ordering, and group-level safety). Prior prototype systems and benchmarks illustrate feasibility but leave open questions about how much deterministic verification and bounded repair alter acceptance rates when the identical initial sample is reused across comparison conditions.

This paper reports a preregistered paired experiment (CAND-2026-001) designed to isolate the marginal effect of a verifier-triggered automated repair on an identical generated candidate. Under shared_initial_candidate semantics the same single initial generation is deterministically reused by both baseline and guarded episodes; any paired difference therefore arises solely from the permitted validator-triggered repair. The primary estimand is the paired difference in per-task binary acceptance by a deterministic verifier (paired_success_rate_difference_guarded_minus_baseline). Because shared_initial_candidate semantics were enforced, this estimand measures a repair-only effect and does not capture potential benefits from independent guarded first-pass generations, constrained prompts, or multi-sample selection.

Contributions. (1) A preregistered, fully archived paired experiment executing 200 intent→configuration tasks with a deterministic verifier and a hosted LLM (gpt-5-mini) under shared_initial_candidate semantics. (2) Complete execution accounting and deterministic reconciliation showing 200 shared initial generations and 1 repair call (201 model calls total). (3) Artifact-grounded confirmatory analysis reporting baseline pass 199/200 and guarded pass 200/200 with contingency counts (n11=199,n10=1,n01=0,n00=0), paired risk difference 0.005, exact two-sided McNemar p = 1.0, and bootstrap 95% CI = [0.00,0.015] (10,000 resamples, seed 1729). (4) A localized failure diagnosis for the single repaired pair using archived validator traces. (5) Detailed reproducibility guidance and bounded limitations tied to archived artifacts and reconciliation records.

II. Related Work

Recent work on LLM-driven NetOps shows convergent interest in intent→configuration synthesis, verification-aware pipelines, and tool-augmented agent designs, but the supplied verified records reveal complementary strengths and limitations that motivated our paired design. NetConfEval and related intent→configuration prototypes demonstrate that LLMs can produce workable device configurations from natural-language intent and that task-style benchmarks are useful for early evaluation; these studies report feasibility and dataset-limited performance measures but do not uniformly integrate a deterministic verifier as the final oracle for acceptance decisions [1]. Deterministic configuration verification methods (e.g., formal and symbolic reachability/policy-check approaches) provide a clear operational oracle for syntactic and semantic invariants and have been developed in the NetOps literature; these verification techniques form the natural oracle used in our experiment and highlight why a verifier-centric evaluation is defensible for configuration-synthesis tasks [2].

Across adjacent domains, generate→validate→repair patterns have empirical support: test-in-the-loop program-repair and synthesis-with-tests pipelines show that validating generated artifacts against executable checks and iteratively repairing failures materially improves correctness in software-engineering tasks [4]. Those demonstrations motivate guarded NetOps pipelines but are not direct evidence of net effects in network-configuration contexts because NetOps invariants and multi-device sequencing introduce distinct dependency

patterns. Tool-augmented agent and model-context approaches further extend capability by granting controlled access to specialized solvers, validators, or execution environments; evaluations of such tool-augmented designs argue they enable more complex, verifiable behavior but also point out engineering and interoperability challenges when integrating heterogeneous oracles [6].

Survey and reporting-guideline literature places additional context around rigor and reproducibility: broad LLM surveys synthesize model capabilities and limitations at scale and underscore the importance of careful experimental reporting and preregistration when evaluating generative systems [5]. Reporting guidelines for studies that use LLMs emphasize explicit documentation of sampling, validation, and analysis choices to avoid overclaiming—this influenced our preregistration and artifact-archiving decisions [3]. Finally, focused work on hallucination detection and validator gating in generative systems highlights common failure modes (confabulation, ordering errors, and adversarially induced fabrications) and motivates verifier gates as necessary but not sufficient mitigations; such analyses support our guarded pipeline as a targeted, bounded intervention rather than a general cure [7].

Synthesis. The supplied and verified literature collectively supports three points: LLM-based intent→config pipelines are feasible and actively prototyped; generate→validate→repair is a promising architectural pattern whose empirical demonstrations to date are stronger in code and testable artifacts than in NetOps-specific verifier studies; and rigorous reporting and verifier-grounded evaluations are necessary to interpret operational claims. Our paper addresses a narrower, previously underexplored empirical gap in the supplied records: a reproducible paired measurement of the marginal (repair-only) effect when the identical initial candidate is reused across baseline and guarded conditions. By enforcing shared_initial_candidate semantics and archiving the verifier traces, the present study isolates that repair-only estimand and complements prior prototype and generate–validate–repair demonstrations [1],[2],[4],[6],[3],[5].

III. Methodology

Preregistration and design freeze. The study design and analysis plan were frozen in preregistration CAND-2026-001 prior to any model calls. The preregistration fixed: (a) a paired binary estimand (paired_success_rate_difference_guarded_minus_baseline) comparing per-task binary verifier acceptance under baseline (no repair) versus guarded (verifier may trigger <=1 repair) conditions; (b) deterministic task selection for N=200 tasks; (c) generation_semantics = shared_initial_candidate (one sampled initial generation per task deterministically reused by both episodes); (d) a maximum of one automated repair call per task; (e) primary failed-call treatment complete_pair_primary and a sensitivity treatment that counts persistent unparsables as failures; and (f) the confirmatory test (exact two-sided McNemar via exact binomial tail) plus a paired-bootstrap procedure for a percentile CI. The preregistration artifact (immutable) is referenced in the execution manifest.

Deterministic task sampling. Tasks were produced by a deterministic task generator (deterministic_netops_task_generator_v5) and selected by a fixed index rule documented in the preregistration. The selection rule is deterministic across the 200-task bank (the preregistration specifies the cycle-and-offset index mapping used to pick each task index); this ensures reproducible task composition and eliminates post-hoc selection. Each manifest encodes: initial_state, expected_final_state, required_sequence (ordered command list), allowed_touched_fields, workflow_policies (group-level constraints, minimum-active rules), and difficulty metadata (changed_interface_count, dependency_rule_count, required_command_count). Difficulty labels used in the study include very_hard and extreme; representative required_command_count values ranged from 3 up to 9 in the archived sample.

Shared_initial_candidate semantics and sampling notes. The adapter contract enforced shared_initial_candidate semantics: for each pair the exact same initial generation (a single sampled candidate) was provided to both the baseline and guarded episodes. Consequently, any observed paired difference can only arise from the action of the deterministic verifier and the single permitted repair; it cannot arise from independent first-pass guarded generations, different sampling calls, prompt-engineering differences, or selection among multiple generated candidates. The model-configuration record declares the requested model as gpt-5-mini and records temperature = null (unset). We explicitly note that temperature = null/unset in the configuration file is not evidence of deterministic sampling and does not imply temperature = 0; nondeterministic hosted-model sampling therefore remains a possible background property even though the experiment used a single initial sample per pair by design.

Generation, sanitizer, and repair policy. For each task the pipeline executed a deterministic formatting sanitizer (automatic_syntax_sanitizer_v1) to correct minor syntactic normalization issues before verifier parsing; this sanitizer is deterministic and is applied prior to scoring per the preregistration. After the shared initial generation and sanitizer, the deterministic verifier parsed the candidate. If the verifier reported any invariant violations, the guarded logic was permitted to invoke at most one automated repair call to the hosted model with a repair prompt constructed from the validator trace (this repair policy and prompt format were specified in the preregistration). Repair calls were counted against the maximum_model_calls budget; the execution manifest reports exactly one repair call across all pairs in the run. The pipeline's missingness plan treated persistent unparsable outputs (after sanitizer and an allowed repair attempt) as scorable failures in the sensitivity analysis; in the primary analysis pairs with complete episodes remained eligible per the preregistered complete_pair_primary rule.

Deterministic verifier as the oracle. The deterministic verifier (deterministic_netops_validator_v5) is the study's opera-

tional oracle. The verifier implements: syntactic parsing into discrete commands; enforcement of allowed_fields and allowed_touched_fields per task manifest; comparison of parsed command sequences against the manifest's required_sequence ordering; evaluation of group-level invariants (e.g., final_group_constraints and minimum_active_in_groups); and a final boolean full_intent_constraint_validation = true only if all encoded constraints are satisfied. The verifier emits structured tracer output for each episode that includes parsed_command_count, required_command_count, observed_touched_field_count, normalized_configuration, validation_before (violation codes and counts), validation_after (post-repair state and validity flag), validator_id and version, and a human-readable description of detected invariant breaches. These tracer fields are the authoritative evidence for per-episode scoring in the analysis.

Paired execution bookkeeping and provider accounting. The preregistered planned episodes were 400 (200 pairs $\times$ 2 conditions). The run completed all planned episodes without terminal provider failures. Provider accounting recorded hosted-model-call events and stage counts consistent with preregistration: the stage counts indicated 200 shared initial generation events and 1 repair event, for a total of 201 model calls. Cache and reuse checks verified that the shared initial candidate was reused per pair and that no cross-condition cache-reuse introduced additional independent generation calls. All runtime provider metadata (call counts, cache hits/misses, stage counts) were reconciled deterministically in the execution reconciliation artifact.

Statistical procedures — confirmatory and CI computation. The primary confirmatory test is the exact two-sided McNemar test computed from discordant pair counts n10 and n01 (where n10 = count of pairs with guarded=1 & baseline=0, and n01 = count with guarded=0 & baseline=1). We implemented the exact two-sided McNemar p-value using the exact binomial-tail formulation: let n = n10 + n01 and k = n10; the two-sided p-value is p = 2 * min{ BinomCDF(k; n, 0.5), 1 - BinomCDF(k - 1; n, 0.5) } with the conventional handling when k = 0. This exact-binomial realization is documented in the analysis log and reproduces the p = 1.0 reported for the observed discordants (n = 1, k = 1). For the paired absolute risk difference we computed a paired-bootstrap 95% percentile confidence interval with 10,000 resamples and bootstrap_seed = 1729; bootstrap parameters and seed are recorded in the analysis log.

Missingness, exclusions, and contamination controls. The preregistration defined deterministic exclusion criteria prior to model calls (e.g., token-normalized identity to public vendor-source templates to avoid leakage) and required logging of any excluded tasks. The run recorded no preregistered exclusions. The sanitizer and missingness policy were applied deterministically; persistent unparsables did not occur in this execution. Contamination checks included token-level similarity scans between generated candidates and vendor-template sources and a contamination summary that would flag unusually high overlaps; no contamination flags were raised in this run.

Reproducibility and artifact grounding. All implementation details required to reproduce the methodology are archived in the execution bundle: the immutable preregistration, the deterministic task generator artifacts and manifests (task bank with required_sequence and workflow_policies), the model-configuration record (including declared model and the explicit temperature = null/unset entry), the verifier implementation and structured tracer outputs, provider-call accounting and deterministic reconciliation, the sanitizer and repair-prompt templates, and the analysis-executor configuration (exact McNemar code path and bootstrap seed/resample settings). The verifier's tracer fields provide the operational link between generated text and the binary scoring used in the confirmatory analysis. Re-running the archived paired analysis executor on the raw results reproduces the contingency counts and CI parameters reported in the paper.

IV. Results

Execution accounting and provider audit. The archived execution manifest records start and end timestamps (UTC) showing the run elapsed $\approx$29 minutes (start 2026-09-01T16:03:05.491039+00:00, end 2026-09-01T16:32:13.298536+00:00). Planned_episode_count = 400 and completed_episode_count = 400; failed_episode_count = 0. Provider-call accounting (deterministic_reconciliation.json) reports hosted_model_call_event_count = 201 with stage_counts = {shared_initial_generation: 200, repair: 1}. Cache_miss_count = 201 and cache_hit_count = 0 indicate fresh model generations for all shared-initial calls and a single repair-stage call; these counts match the preregistered shared_initial_candidate semantics and the recorded single repair invocation.

Primary confirmatory outcomes. analysis/condition_summary.csv shows baseline successes = 199 and failures = 1 (baseline success_rate = 0.995) and guarded successes = 200 and failures = 0 (guarded success_rate = 1.000). The paired contingency table (analysis/paired_contingency_table.csv) records n11 = 199, n10 = 1, n01 = 0, n00 = 0. From these counts the paired absolute risk difference (guarded - baseline) = 0.005. Using the exact two-sided McNemar formulation described in Methods, with n = n10 + n01 = 1 and k = n10 = 1 we compute BinomCDF(1;1,0.5) = 1 and 1 - BinomCDF(0;1,0.5) = 0.5; hence p = 2 $\times$ min(1, 0.5) = 1.0. The paired-bootstrap percentile 95% confidence interval for the absolute risk difference was computed with 10,000 resamples and bootstrap_seed = 1729 (analysis/analysis_log.jsonl) and equals [0.00, 0.015]. These numeric artifacts (analysis/condition_summary.csv, analysis/paired_contingency_table.csv, analysis/analysis_log.jsonl) are archived and reproduce the reported confirmatory values when the paired_binary_analysis_v1 executor is re-run on the archived input_results_sha256.

Single repair event and diagnostic traces. The guarded pipeline invoked exactly one automated repair. Archived validator traces for the repaired pair indicate an extreme com-

posite task (difficulty metadata recorded in the task manifest: required_command_count = 9, changed_interface_count = 3, difficulty.level = "extreme") whose shared initial candidate violated the task's required_sequence ordering constraint. The verifier trace for that episode documents validation_before.violation_codes showing the ordering breach, the shared_initial_candidate (the identical candidate reused by baseline and guarded), and validation_after with validation_after.valid = true once the bounded repair was applied. The repair prompt/trace and the validator's normalized_configuration show the localized reordering the repair enacted; after repair the scorer returned score = 1 (scoring_reason_code = "VALID_CONFIGURATION"). These per-episode fields are present in the execution/scoring/* artifacts and constitute the archived evidence that the single observed paired improvement resulted from a verifier-triggered reordering repair that converted a sequenced-dependency failure into a valid configuration under the verifier's invariants.

Missingness, exclusions, and contamination checks. The run recorded no preregistered exclusions and no persistent unparsables; analysis/missingness_summary.csv reports complete_pairs = 200 and zeros for all other missingness categories. analysis/contamination_summary.csv is empty (no flagged contamination). Deterministic_reconciliation.json indicates pair_accounting_consistent = true and all_deterministic_consistency_checks_passed = true. No episodes have terminal_error_type set, and provider-call audit details align with preregistration (200 shared initial generations, 1 repair). These artifacts support the statement that the confirmatory analysis included the full preregistered sample without post-hoc exclusions.

Exploratory diagnostics by difficulty metadata. Representative task manifests archived in execution/task_manifest.jsonl encode difficulty metadata (changed_interface_count, dependency_rule_count, required_command_count). In the archived representative subset required_command_count values span 3–9 and changed_interface_count values up to 3; the lone baseline failure is associated with the extreme composite difficulty pattern (highest required_command_count in the sample), consistent with the preregistered exploratory hypothesis that multi-device sequencing and dependency complexity concentrate residual failures. analysis/paired_difference_figure.svg (archived) visualizes condition rates and the bootstrap distribution used for the CI; those visual diagnostics and the per-task validator_trace fields (parsed_command_count, observed_touched_field_count, violation_codes) are available for targeted reanalysis to inspect whether other high-difficulty tasks produced near-miss traces that would benefit from alternative repair strategies.

Synthesis of quantitative uncertainty. The realized baseline success rate (199/200 = 99.5%) places the confirmatory test in a sparse-discordant regime: with only one discordant pair the exact McNemar p-value gives limited directional evidence (p = 1.0), while the paired-bootstrap CI [0.00, 0.015] quantifies the upper bound of plausible absolute improvement attributable to repair in this repair-only estimand. In practical terms, these results show that, for this synthetic verifier-backed task bank and the single-model run, bounded verifier-triggered repair corrected one expressible sequencing fault; they do not imply large marginal gains across broader settings without further experiments (e.g., independent guarded-generation contrasts, multi-model studies, or live dataplane emulation).

V. Discussion

We expand the interpretive analysis and diagnostics here while preserving the paper's existing supported content.

Estimand interpretation and scope (explicit). Because the experiment used shared_initial_candidate semantics, baseline and guarded episodes for each task began from the identical sampled candidate; consequently the primary estimand intentionally measures the marginal effect of permitting an automated verifier-triggered repair on that identical candidate (a repair-only effect). This design isolates repair mechanics from any potential benefits that could arise if the guarded condition were allowed to sample independently or draw multiple candidates.

Sparse-discordant regime and inferential consequences. The observed data (n11=199, n10=1, n01=0) place the analysis in a sparse-discordant regime, a setting where conventional asymptotic approximations are unreliable and exact discrete tests or resampling are appropriate. Our use of the exact binomial-tail McNemar calculation and a paired-bootstrap percentile CI (10,000 resamples, seed 1729 as archived in analysis/analysis_log.jsonl) is consistent with best practice for small discordant counts. Practically, the realized baseline success rate (99.5%) substantially reduced the experiment's sensitivity to detect modest repair-only effects relative to our preregistered power target (designed for mid-range baseline rates ~45–65%). The bootstrap CI [0.00, 0.015] quantifies that the data are compatible with effects up to 1.5 percentage points in absolute terms under our paired repair-only estimand; the preregistered design could not reliably detect larger effects because none were present.

Mechanistic diagnostic of the repaired episode. The single repair call corrected a concrete sequencing violation expressible in the task manifest's required_sequence field (required_command_count = 9 in the repaired manifest). Archived validator_trace fields show validation_before.violation_codes listing the ordering breach, parsed_command_count equal to the presented commands, and validation_after.valid = true after the automated reordering repair. These per-episode traces demonstrate: (1) the verifier's capacity to detect ordering invariants encoded in the manifest; (2) the repair module's ability to perform localized, semantics-preserving reorderings that the verifier accepts; and (3) that such faults concentrated in tasks with higher required_command_count and extreme difficulty labels in our deterministic task bank. All diagnostic fields cited above are available in execution/scoring/task-00000X-*.json in the archived bundle.

Construct validity of the deterministic verifier. deterministic_netops_validator_v5 enforces a structured subset of op-

erational invariants encoded in the task payloads (ordering, allowed_touched_fields, group-level up/down constraints, and explicit per-field prerequisites). This verifier therefore provides a high-fidelity oracle for manifest-encoded invariants but does not emulate dataplane behavior, vendor runtime idiosyncrasies, or external control-plane side-effects. Construct-valid conclusions should therefore be framed strictly in terms of manifest-specified, deterministically verifiable invariants; the experiment does not claim to validate dataplane reachability or vendor CLI execution semantics that lie outside the verifier's encoding.

Design implications for future NetOps evaluations. The paired shared_initial_candidate design cleanly isolates repair capability; however, operational questions remain that require alternative designs: (a) independent guarded-first-pass sampling would quantify whether guarded prompting or constrained generation yields a higher baseline of valid samples before repair; (b) multi-sample selection (n-best) could reveal selection-improvement effects; and (c) multi-model comparisons would quantify model-dependent failure patterns. From an experimental-planning perspective our results highlight that preregistered power calculations must account for plausible high baseline performance—if baseline rates turn out near 100% the discordant-pair approach will often be underpowered to detect small absolute repairs-only gains.

Reproducibility, artifact usage, and recommended reanalyses. All artifacts necessary to reproduce analysis are archived: execution/task_manifest.jsonl (deterministic task selection and required_sequence fields), execution/model_configuration.json (declared model, noting temperature = null/unset), execution/responses/ (shared-initial and per-condition responses), execution/scoring/ (per-episode validator traces), analysis/paired_contingency_table.csv and analysis/condition_summary.csv, deterministic_reconciliation.json (provider accounting and stage_counts), and analysis/analysis_log.jsonl (bootstrap parameters and seed). Re-running the paired_binary_analysis_v1 executor on the archived input_results_sha256 reproduces the confirmatory results. For targeted reanalysis, researchers can (i) recompute bootstrap intervals under alternative resampling schemes, (ii) treat the single repaired task as an outlier to study robustness, or (iii) reclassify manifest invariants to evaluate sensitivity of verifier coverage—all without altering original model-call artifacts.

Threats to validity revisited (artifact-grounded). Internal validity: shared_initial_candidate semantics intentionally eliminate differences due to separate sampling, but they also prevent measuring effects of guarded prompt variants; model sampling nondeterminism remains possible because temperature was unset (execution/model_configuration.json records temperature = null). Construct validity: the verifier enforces only encoded invariants; omissions in the manifest (for example, dataplane-dependent invariants) are not tested. External validity: a single hosted model (gpt-5-mini) and a synthetic deterministic task bank limit generalizability; results do not imply identical outcomes for other models, live devices, or telemetry-driven validation. Conclusion validity: with one discordant pair the statistical evidence for a nonzero repair-only effect is weak (p = 1.0), though the bootstrap CI constrains plausible magnitudes.

Operational implications and cautious hypotheses. Artifact-grounded evidence demonstrates that bounded verifier-triggered repairs can fix localized ordering violations that are detectible and expressible in task manifests. From an operational standpoint this suggests a practical pattern: require explicit, machine-checkable manifest invariants for change tasks and pair LLM synthesis with deterministic validation/repair gates before human or automated deployment. We emphasize these are actionable hypotheses supported by the archived diagnostics in this controlled synthetic setting; they are not operational claims that bounded repair will resolve dataplane-level or adversarially induced failures without further evaluation.

Summary of expanded diagnostics. The archived execution bundle provides the evidentiary chain: deterministic task manifests encoding the invariants, the shared initial generated candidates, per-episode validator traces showing before/after states, provider-call accounting confirming shared_initial_generation = 200 and repair = 1, and analysis artifacts (contingency table, bootstrap parameters) documenting the statistical regime. Together these artifacts ground the narrower, repair-only conclusion and illuminate concrete design decisions for future, broader evaluations (independent guarded sampling, multi-model studies, live emulation).

VI. Limitations

- Synthetic deterministic task bank and verifier: results reflect correctness under the chosen synthetic intent templates and deterministic verifier and do not guarantee similar performance on live heterogeneous production devices or telemetry.
- Single-model experiment (gpt-5-mini): findings do not demonstrate multi-model generality.
- Shared_initial_candidate semantics: baseline and guarded conditions began from the same initial generation; the study measures verifier/repair effects only and does not evaluate independent first-pass generation contrasts.
- Limited adversarial/contamination testing: the run did not include systematic adversarial retrieval or poisoned-context attacks; such threats remain an open evaluation axis.
- Oracle coverage: the deterministic verifier enforces encoded invariants but may omit runtime behaviors and device-specific semantics observable only through live execution; external validity to live deployments is therefore limited.

VII. Conclusion

We executed a preregistered paired experiment measuring the marginal effect of a verifier-triggered repair under shared_initial_candidate semantics for LLM-generated network configurations. In 200 synthetic deterministic tasks

with gpt-5-mini and deterministic_netops_validator_v5, baseline acceptance was 199/200 and guarded acceptance was 200/200; a single automated repair corrected a multi-device ordering violation. Paired absolute risk difference = 0.005 with paired-bootstrap 95% CI [0.00, 0.015] (10,000 resamples, seed 1729) and exact two-sided McNemar p = 1.0. Artifact-grounded evidence therefore supports a constrained conclusion: verifier-guarded bounded repair can correct localized ordering faults in this synthetic verifier-backed setting. Broader operational claims require additional experiments: multi-model studies, independent-first-pass guarded semantics, adversarial contamination testing, and live-device or dataplane emulation to capture runtime semantics not modeled by the deterministic verifier.

Disclosure Statement

Disclosure: This run implemented preregistered study CAND-2026-001 and was executed autonomously after the autonomy lock. Declared model: gpt-5-mini (execution/model_configuration.json records temperature = null/unset; null/unset is not evidence of deterministic sampling or temperature = 0). Orchestration adapter: hosted_netops_gvr_v1. Exact initial master prompt archived at provenance/master_prompt.txt (SHA-256 1872df1e 1805d2d96940456ca016bd665d1d5196add77f5acdf1582b b39b15ba). Preregistration fixed generation_semantics = shared_initial_candidate (one sampled initial generation per task was deterministically reused by baseline and guarded episodes) and allowed at most one automated repair call per task. Model-call accounting in the archived execution manifest reflects 200 shared initial generation calls and 1 repair call (201 model calls total). The deterministic verifier used was deterministic_netops_validator_v5. Humans selected the broad topic family and constrained the initial master prompt prior to the autonomy lock as permitted by the track; no human manually edited technical manuscript content after the autonomy lock. All provider traces, verifier logs, task manifests, model-configuration records, and analysis artifacts are archived in the execution bundle and indexed by the execution manifest.

References

[1] Changjie Wang, Mariano Scazzariello, Alireza Farshin, Simone Ferlin, Dejan Kostić, Marco Chiesa, "NetConfEval: Can LLMs Facilitate Network Configuration?", 2024, 10.1145/3656296

[2] Ryan Beckett, Aarti Gupta, Ratul Mahajan, David Walker, "A General Approach to Network Configuration Verification", 2017, 10.1145/3098822.3098834

[3] Jack Gallifant, Majid Afshar, Saleem Ameen, Yindalon Aphinyanaphongs, Shan Chen, Giovanni Cacciamani, Dina Demner-Fushman, Dmitriy Dligach, Roxana Daneshjou, Chrystinne Oliveira Fernandes, Lasse Hyldig Hansen, Adam Landman, Lisa Soleymani Lehmann, Liam G. McCoy, Timothy A. Miller, Amy C. Moreno, Nikolaj Munch, David Restrepo, Guergana Savova, Renato Umeton, Judy Wawira Gichoya, Professor Gary S. Collins, Karel G.M. Moons, Leo Anthony Celi, Danielle S. Bitterman, "The TRIPOD-LLM reporting guideline for studies using large language models", 2025, 10.1038/s41591-024-03425-5

[4] Yunhe Li, "Test-in-the-loop LLM Repair: Verifiable Automated Program Repair on QuixBugs with a "Failing Test → Patch → Regression Test" Loop", 2024, 10.69987/jacs.2024.40206

[5] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Zican Dong, Yupeng Hou, Beichen Zhang, Yingqian Min, Junjie Zhang, Peiyu Liu, Xiaolei Wang, Yifan Du, Yushuo Chen, Yushuo Chen, Zhipeng Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Peiyu Liu, Yiwen Hu, Jian-Yun Nie, Ji-Rong Wen, "A Survey of Large Language Models", 2026, 10.1007/s11704-026-60308-3

[6] Haoran Dong, "Tool-Augmented Large Language Model Agents for Analysis: A Focused Study", 2026, 10.2139/ssrn.6647800